POLYGLOT WORKSHOP

WORKING WITH TEXT DATASETS

PART II

Tutors
Shiza Fatimah & Nicholas Kluge

TRA SUSTAINABLE FUTURES

UNIVERSITÄT BONN

AGENDA

1. Tokenization Algorithms

- ✓ Tokenization Basics
- ✓ Subword Tokenization Algorithms
- ✓ Training a Tokenizer

2. Evaluating Tokenizers

- ✓ The UNK Counter
- ✓ Subword Fertility
- ✓ Proportion of Continued Words

3. The Performance Paradox

- ✓ What are ablations?
- ✓ How to ablate?

AGENDA

4. Tokenization (Pretraining)

- ✓ Formatting
- ✓ Packing
- ✓ Masking
- ✓ Caching

TOKENIZATION ALGORITHMS

Encoding Language in a Machine-Friendly Format

TOKENIZATION BASICS

Tokenization is a **core component** of the LLM development pipeline. Tokenizers serve one purpose: to **convert text into a representation that the model can process. i.e.,** the process of breaking text into smaller units, or tokens, such as words, subwords, or characters.

- ✔ **Sentence:** I love avocados.
- ✔ **Tokens:** (“I”, 17), (“love”, 23), (“avocados”, 42), (".", 5)
- ✔ **Token IDs:** (17, 23, 42, 5)
- ✔ **Embeddings:** ([0.1, 0.32, 0.5, 0.2], [0.7, 0.24, 0.5, 0.9], [0.31, 0.7, 0.07, 0.89], [0.42, 0.28, 0.88, 0.1])

>> Highly recommended lecture: *Let's build the GPT Tokenizer* (🔗).

>> Daniel Jurafsky & James H. Martin. (2026). *Words and Tokens* (🔗).

TOKENIZATION BASICS

- ✓ One of the most hellish aspects of tokenizers is **how many knobs there are to tweak**, and every one of them **directly affects model performance**. Imagine training an otherwise highly capable model that, for example, cannot reliably produce code blocks (“```”).

- ✓ In this sense, it is truly infuriating how **silently** tokenizers will fail: you may only realize something is wrong when you start doing qualitative *“vibe checks”* on model outputs. **By that point, fixing the issue usually means retraining the model from scratch, since the tokenizer cannot be trivially swapped out.**

Changing the tokenizer changes the embedding mapping, which effectively means you “no longer have the same model at all” (not fully true, but we will talk more about this later).

TOKENIZATION BASICS

- ✓ *“How good is a tokenizer?”* is a difficult question to answer. Before attempting to answer it, we should first understand the problem a tokenizer is meant to solve: **compressing information while (attempting to) preserve meaning**—that is, encoding.
- ✓ Encoding is the process of **converting data from one form into another**. Most tokenization algorithms in use today were originally proposed for **encoding and compression**.
- ✓ Perhaps the simplest tokenizer to consider is a **character-based** one, in which each character is treated as a separate token.

“I love avocados.” → [‘I’, ‘ ‘, ‘l’, ‘o’, ‘v’, ‘e’, ‘ ‘, ‘a’, ‘v’, ‘o’, ‘c’, ‘a’, ‘d’, ‘o’, ‘s’, ‘.’]

Question: What are the PROs and CONs of this approach?

TOKENIZATION BASICS

- ✓ A second idea that often comes to mind is **word-based tokenization**. For languages written in the Latin script, this approach is generally easy to implement with only a few simple rules and often yields reasonably good results.

“I love avocados.” → [‘I’, ‘love’, ‘avocados.’]

Question: What are the PROs and CONs of this approach?

TOKENIZATION BASICS

- ✓ Another approach is **byte-level tokenization**, typically built on top of Unicode encodings. This method treats each byte as a separate token, providing a highly general mechanism for handling text regardless of language or script.

- ✓ **Unicode** is *"the universal character encoding, designed to support the worldwide interchange, processing, and display of the written texts of the diverse languages and technical disciplines of the modern world."* In practice, it covers virtually **all natural languages, as well as many non-spoken symbol systems such as musical notation and mathematical symbols**.

- ✓ That said, it is not *truly* universal. Languages like **Tengwar** or **Klingon** remain unofficially supported.

"I love avocados." → \x49 \x20 \x6c \x6f \x76 \x65 \x20 \x61 \x76 \x6f \x63 \x61 \x64 \x6f \x73 \x2e

Question: What are the PROs and CONs of this approach?

TOKENIZATION BASICS

- ✓ Before advancing to tokenization algorithms, we need to talk about pre-tokenization (**the preparation of the text before it can be encoded**). **This can truly break your tokenizer.**

Question: What would happen if we trained a tokenizer with text that has been half normalized by one method (NFKC) and half normalized by another method (NFC)?

TOKENIZATION BASICS

Unicode normalization (NFC, NFKC, NFD, etc.) determines how sequences of code points are represented:

- ✔ **NFC:** Composes characters where Possible, e.g., “é” as a single code point (U+00E9).
- ✔ **NFD/NFKD:** Decomposes characters, e.g., “é” becomes “e +´” (two code points).

If two texts normalize differently, the same *visual te*xt can produce **different token sequences.**

>> Always normalize the **entire corpus consistently** before training a tokenizer.

>> **NFKC** can simplify compatibility symbols and is generally more useful for multilingual or web text.

>> Most libraries used to train tokenizers either pre-normalize the corpus or force you to set up a normalizer, i.e., **prevent you from shooting yourself in the foot.**

SUBWORD TOKENIZATION ALGORITHMS

Subword tokenization is like giving words a **Lego** makeover. It breaks them into smaller, reusable pieces called ***subwords***. This trick **solves the headaches of traditional word-based tokenization**, such as out-of-vocabulary words (“<|UNK|>”) and very **large vocabularies**. The principle behind this approach is simple. Common words stay whole (i.e., **combinations of bytes are minted into a single token**), while rare or **unusual words get chopped into meaningful building blocks**.

"I love tokenization." → [‘I’, ‘love’, ‘token’, ‘ization’, ‘.’]

The subword **“ization”** is a common suffix in English (because English is an agglutinative language), and by splitting it off, the tokenizer can more easily handle variations of words that have that same suffix (e.g., “initialization”, “finalization”, “optimization”).

SUBWORD TOKENIZATION ALGORITHMS

The most standard subword tokenization algorithms are **Byte Pair Encoding** (BPE), **WordPiece**, and **Unigram Language Model** (UniLM):

Algorithm	Base Idea	Vocabulary Growth	Merge/Removal Criteria	Notes
BPE	Merge the most frequent pairs	Start small, grow by merges	Most frequent consecutive token pair	Simple, used in GPT; byte-level BPE handles all Unicode
WordPiece	Merge pairs considering likelihood	Start small, grow by merges	Frequency of pair ÷ (freq of part1 × freq of part2)	Prioritizes less common subwords; used in BERT
Unigram (UniLM)	Probabilistic token removal	Start large, shrink by removing low-impact tokens	Loss increase if token removed	Tokens treated independently; uses Viterbi for best segmentation

TOKENIZATION LIBRARIES

When it comes to choosing a library to train/run a tokenizer, based on my experience, there are only a couple of choices to consider, and I will mention only three (two I actually used in real production):

Library	Can Train Tokenizers?	Supported Algorithms	Primary Strengths	Typical Use Cases
SentencePiece	✔	BPE, Unigram, Char/Word	Fast (C++), Battle Tested, Language Agnostic	Training Tokenizers for Custom Use
Tokenizers	✔	BPE, WordPiece, Unigram	Also Fast (Rust), Very Modular	Training & Production
tiktoken	⚠ Limited	BPE-style	Optimized for OpenAI's models	Couting Tokens?

TRAINING A TOKENIZER

"Why would I want to train a tokenizer? Can't I just use one that already exists?"

Yes, you can. In fact, **I recommend.** Dealing with tokenizers is **incredibly annoying**.

The advantages:

- ✔ **You won't shoot yourself in the foot (but someone else might**...)
- ✔ Currently, there's a good selection of strong tokenizers available (GPT-OSS, Qwen, Llama, etc.).
- ✔ Qwen excels particularly in some low-resource languages.

The disadvantages:

- ✘ **Lack of representation = bad compression** (Petrov et al. 2023) **(cool visualization →** 🧶**)**.
- ✘ Large vocabularies = **Huge logits tensor to materialize** (batch*seq_len, vocab_size).
- ✘ Pruning is an option, but I don't particularly know of any good "standard" practices for pruning.

***"Tokenizer transplantation"* is a promising merging approach. Requires a model trained with the tokenizer you would like to keep (**🧶**).**

TRAINING A TOKENIZER

"But I want to train my own tokenizer!" **Okay. You have been warned. Here are the main things you should pay attention to:**

The Data:

- ✔ Unlike pretraining a model, **you don't need much data for this**.
- ✔ There is **no standard amount to use. Enough to fill your vocabulary** (e.g., 100K, 500K, 1M, 2M)
- ✔ Pass a certain point, and **more data is just a waste of time**.
- ✔ It should be a **diverse representation** of the domain you want to.
- ✔ All text should be encoded with the **same normalization scheme**.

TRAINING A TOKENIZER

The Vocabulary Size:

- ✔ **Optimal vocabulary** sizes for monolingual models are around **30K-50K tokens** (🤗).
- ✔ **Multilingual** models require **extended** vocabularies (**100K-200K**)
- ✔ We should **ALWAYS** aim for vocab sizes that **produce tensors the GPU will like**.
- ✔ **Padding** the embedding layer/tokenizer is a way to **avoid having weird dimensions**.
- ✔ The bigger the vocabulary, the **more compression**.
- ✔ The bigger the vocabulary, the **bigger the memory requirements** to materialize the logits.

Must read: Ali et al. (2023). *Tokenizer Choice For LLM Training: Negligible or Crucial?*

TRAINING A TOKENIZER

The Special Tokens:

- ✔ **Beginning-of-Sentence, End-of-Sentence, Unknown, Padding.**
- ✔ **Other demarcations that you might want to use (e.g., Chat, FIM, Tool Usage).**
- ✔ **Attention Sinks and the Beginning-of-Sentence token (🧶).**
- ✔ Special tokens can be used to **handle domain-specific patterns (🧶).**

“<BOS>”	“Token”	“ization”	“_is”	“_a”	“pain”	“_in”	“_the_”	“<UNK>”	“!”	“<EOS>”	“<PAD>”	“<PAD>”	“<PAD>”	“<PAD>”	“<PAD>”

TRAINING A TOKENIZER

GPT-2

```
def fibRec(n):↩
    if n < 2:↩
        return n↩
    else:↩
        return fibRec(n-1) + fibRec(n-2)
```

55 tokens

GPT-NeoX-20B

```
def fibRec(n):↩
    if n < 2:↩
        return n↩
    else:↩
        return fibRec(n-1) + fibRec(n-2)
```

39 tokens

GPT-NeoX-20B: An Open-Source Autoregressive Language Model

POLYGLOT

TRAINING A TOKENIZER

Handling Numbers:

- ✔ Most modern tokenizers do **single-digit splitting** (🔗, 🔗).
- ✔ Llama3 (🔗) **encodes numbers from 1 to 999** as unique tokens.
- ✔ Hard to say which strategy **works the best for mathematical reasoning**.

Must read: *How would you tokenize (or break down) a million digits of pi?*

Must read: *Integer tokenization is now much less insane.*

GPT-4

```
3.14159265358979323846264338
32795028841971693993751058209
74944592307816406286208998
62803482534211706798214808 6
51328230664709384460955058 2
23172535940812848111745028 4
10270193852110555964462294 8
```

LLAMA 3

```
3.14159265358979323846264338
32795028841971693993751058209
74944592307816406286208998
62803482534211706798214808 6
51328230664709384460955058 2
23172535940812848111745028 4
10270193852110555964462294 8
```

CLAUDE

```
3.14159265358979323846264338
32795028841971693993751058209
74944592307816406286208998 6
28034825342117067982148086 5
13282306647093844609550582 3
17253594081284811174502841027
01938521105559644622948954930
```

GEMMA

```
<bos>3.14159265358979323846264
33832795028841971693993751058
20974944592307816406286208998
62803482534211706798214808 65
13282306647093844609550582231
72535940812848111745028410270
19385211055596446229489549303
```

How would you tokenize (or break down) a million digits of pi?

POLYGLOT

TRAINING A TOKENIZER

If you're using SentencePiece:

- ✔ Setting the training (multi-thread support!) is **simpler**.
- ✔ It's recommended to **convert it to an HF Tokenizer**.
- ✔ Having multiple tokenizers that behave the same across platforms is an **unnecessary complication**.

If you're using Tokenizers:

- ✔ You will have to **set all components** that make up a tokenizer (**or train it from a template**).
- ✔ This gives more **flexibility**, but also **more ways to shoot yourself in the foot**.

TRAINING A TOKENIZER

Component	Role in Tokenization	What It Does	Effect
Normalizer	Pre-processing	Cleans/normalizes raw text before splitting	Unicode normalization (NFC/NFKC), lowercase, strip accents
Pre-Tokenizer	Pre-processing	Breaks text into rough chunks (words, punctuation)	Whitespace or byte-level splits before model rules apply
Model	Core Tokenization Algorithm	Applies learned/token rules to map text pieces to token IDs	Subword algorithms (BPE, WordPiece, Unigram)
Post-Processor	Adjustments and Formatting	Adds special tokens or templates for model inputs	Inserts tokens like [CLS], [SEP] sequences
Decoder	Reverse mapping	Converts token IDs back to readable text	Reverts byte-level, metaspace, or WordPiece artifacts

Tokenizers Documentation: Components.

TRAINING A TOKENIZER

It is always safer when you don't know what you are doing to try to emulate something that has already worked (📎).

- ✔ Learn how to set up and train a subword tokenizer using SentencePiece or Tokenizers.
- ✔ Understand the impact of different parameters on the tokenizer's behavior.
- ✔ Vibe test the trained tokenizer on sample text inputs.

EXERCISE

EVALUATING TOKENIZERS

What makes a tokenizer efficient?

EVALUATING TOKENIZERS

We have just trained our tokenizer and would like to see how it performs using a ***"goodness"* metric**. Moreover, we would like to **compare** it with other tokenizers to further **justify the benefits** (if any) of using a custom tokenizer over a battle-tested alternative.

While **vibe checks are essential** for uncovering subtle bugs and formatting issues, **we cannot simply eyeball a few tokenization examples and call it good**, any more than we can make architectural changes based on intuition without running ablations.

QUESTION: *How would you **evaluate** a tokenizer? Can you provide **concrete metrics** to assess tokenizer performance?*

THE UNK COUNTER

One simple way to evaluate a tokenizer is to **count the number of unknown tokens it generates** (or their proportion). A tokenizer that is **unable to represent a lot of the symbols that are used in a given domain** is **not a very good tokenizer** (e.g., n of UNKs / total tokens).

Thanks to byte-level encoding (like BPE), UNKs are something you won't have to worry about.

SUBWORD FERTILITY

Subword fertility measures **the average number of tokens needed to encode a word**. Lower fertility means better compression, which translates to faster training (i.e., **you can pack more tokens in the batch**) and inference (i.e., **model generation, in terms of words and chars, is faster**) (🖇️).

The standard approach to measuring fertility is to calculate the words-to-tokens ratio (word fertility), which quantifies the average number of tokens per word (n tokens / n words). **A theoretical minimum of 1 would mean the tokenizer's vocabulary contains every word in the reference text**.

QUESTION: *Why words? Why not a* ***characters-to-tokens*** *ratio or a* ***bytes-to-tokens*** *ratio?*

SUBWORD FERTILITY

- ✓ Bytes can be skewed since **characters in different scripts require different byte representations** (e.g., Chinese characters use three bytes in UTF-8 while Latin characters use one to two).

- ✓ Similarly, using the number of characters doesn't account for the fact that words vary dramatically in length across languages. For instance, **Chinese words tend to be much shorter than German compound words**.

- ✓ However, how to segment words across languages is something that is not so simple and **requires domain knowledge to do it right**.

>> Cool resource to explore how fertility varies across tokenizers → (🖇️)

PROPORTION OF CONTINUED WORDS

The ratio of “real” text words encoded with 2 tokens or more. Measures **how often a tokenizer splits words**. A value of 0 means that the tokenizer never splits and 1 that it always splits (🖇).

IMPORTANT: Special tokens need special treatment when running these kinds of evaluations (SF and PCW). A tokenizer that is **incapable of tokenizing most of the text** (i.e., treating everything as an UNK) **will have very low SF and PCW**, but for the wrong reason. Also, if the tokenizer **always adds a BOS token to the beginning of a sentence**, and you do not disregard this when calculating something like PCW, by necessity, you will get a result of 1 (i.e., **there is always more than one token for every word, because we are always adding BOS!**)

TRAINING A TOKENIZER

These metrics help us make **more informed decisions about how to encode the data used to train our models**. **Without them, we are even more in the dark.**

- ✔ Learn how to **evaluate a subword tokenizer** using different metrics.
- ✔ **Compare** the performance of different tokenizers.
- ✔ Understand the implications of **tokenizer performance on the costs of training language** models.

EXERCISE

THE PERFORMANCE PARADOX: AN ABLATION MYSTERY

Better Compression ≈ Better Performance, right?

ABLATIONS

What is this word "ablation"?

- ✓ The term *ablation* is credited to **Allen Newell**, one of the founders of the field. It's borrowed by analogy from *ablation* in biology, where you **remove part of a system to see what changes**.

- ✓ The motivation is simple: if you want to **understand how much a specific component contributes to measurable quantity X, you remove that component and observe how X changes**.

- ✓ Designing ablation experiments is a **huge part of what we do**! An LLM development or research pipeline **without ablations is basically a shot in the dark** (whether well-informed or not). That's also why developing an LLM usually **requires far more compute** than just the single *"final"* training run you care about.

HOW TO ABLATE

- ✓ Good ablations compare changes that are (to the extent of what is possible and practical) ***isolated***. **If you change two things at once, that's not an ablation!** You won't be able to tell what actually caused the effect you observed. The pipeline should always be: **hold everything else constant and change one variable** (e.g., data mixture, learning rate, batch size, model depth), **measure the effect**.

- ✓ With that in mind, we're going to do a bit of **make-believe in the next exercise**. You'll play the **role of a research scientist analyzing evaluation and ablation results**, and you'll try to come up with **explanations, hypotheses, and concrete ways to test those hypotheses**.

AN ABLATION MYSTERY

“You're tasked with selecting a tokenizer for training a language model. Your team has invested significant resources in training a custom tokenizer, and you need to justify your choice based on evidence. According to conventional wisdom, better compression metrics should lead to more efficient training and better downstream performance. You have tested your tokenizer against a strong baseline using standard intrinsic metrics, such as the ones we just covered (e.g., fertility, PCW). Here are the results you obtained …”

- ✔ Develop **critical thinking** about evaluation metrics.
- ✔ Learn to distinguish between **intrinsic and extrinsic evaluation**.
- ✔ Practice data analysis and **hypothesis formation**.

EXERCISE

TOKENIZATION FOR PRETRAINING

Formatting. Packing. Masking. Caching.

TOKENIZATION FOR PRETRAINING

We have **collected** and **filtered** the data. **Created** it. We learned how to **train tokenizers** for encoding our data. We learned how to **evaluate** if we did a good job. Now, we need to convert all of this into the actual training dataset, i.e., **the tokenized version of our dataset(s)**.

Our objective now is to convert raw text into **fixed-length token sequences that can be efficiently streamed to the model during training**.

- ✔ Formatting.
- ✔ Packing.
- ✔ Masking.
- ✔ Caching.

"Everything that can be done offline should be done offline."

FORMATTING

For causal language modeling (**CLM**), the training data consists of a **single long sequence of tokens**. Each sequence is capped at a **maximum context length**. At this stage, each document is represented as **a vector of token IDs**.

- ✔ Typically **2048–8192 tokens** (longer contexts come later).
  - ✔ **Inputs:** token IDs ([1,2,3]).
  - ✔ **Labels:** same sequence, shifted **one token to the right** ([2,3]) → **N – 1 predictions.**
  - ✔ **Attention mask:** Binary mask (1s and 0s) → **all ones.**

In some cases (e.g., Transformers). You only need to store **the token IDs**. Labels and attention masks can be generated on the fly. **This saves storage and simplifies datasets.**

PACKING

Documents rarely match the context length exactly. Some documents are **shorter** than the context, while others are **longer** than the context. The solution is **to pack, i.e., concatenate** multiple documents into a single sequence. Pretraining operates at a **massive scale**. Not packing would result in significant compute waste, and relying solely on ***"perfectly sized"*** documents is unrealistic.

- ✔ Documents are separated by an **EOS token**.
- ✔ Anything beyond the max context length is **truncated**.

This naive packing strategy works well for **pretraining**. For **SFT, FIM, or instruction tuning**, packing must be handled more carefully because formatting and boundaries matter more than token count.

MASKING

Masking, in this context, means excluding tokens either from the **loss** or the **attention mechanism**.

Common cases:

- ✔ Padding / EOS / BOS tokens.
- ✔ Masked from loss (-100 in PyTorch).
- ✔ Important clarification:

  >> Masking a token from the **loss** does **not** stop its embedding from being updated.

  >> **Gradients still flow** through masked tokens.

MASKING

Advanced Cases:

- ✔ Cross-Document Attention (**4D attention masking**).
- ✔ Prevents tokens from Document A attending to Document B within a packed sequence.
- ✔ Improvements are more apparent when **dealing with long contexts**.
- ✔ Natively supported via Transformers and Nanotron, but **out of scope for this workshop**.

CACHING, STORAGE, AND METADATA

✔ How to smartly store and cache your files is also an important consideration. In the setup we have at Marvin, for example, we store large datasets (tokenized or not) on **scratch** (one of the partitions users have access to in the Lustre system), which is **optimized for large, long-lived files, and uses HDDs for storage**.

✔ Meanwhile, any cache that helps files load faster is saved in the **NVMe/SSD** portion of Lustre. This helps **speed up training startup and avoid cold-start delays**.

💡 **Always save metadata alongside your tokenized dataset** (number of samples, token count, tokenizer path, number of documents, etc.). This helps us avoid reading the entire dataset to compute statistics. **When you are doing multi-stage training with different data mixtures, you will want fast access to this information**.

TOKENIZATION FOR PRETRAINING

- ✔ Learn how to set up a **tokenization pipeline** using the Datasets library.
- ✔ Learn how to **format a text dataset** for pretraining a language model.

EXERCISE

AGENDA FOR TOMORROW

LLM PRETRAINING 101

Nicholas Kluge & Aniket Sen

Polyglot team

08:00 — 12:00

POST-TRAINING AND ALIGNMENT

Florian May & Nicholas Kluge

Lamarr Institute / Polyglot team

13:00 – 17:00

SEE YOU ALL TOMORROW